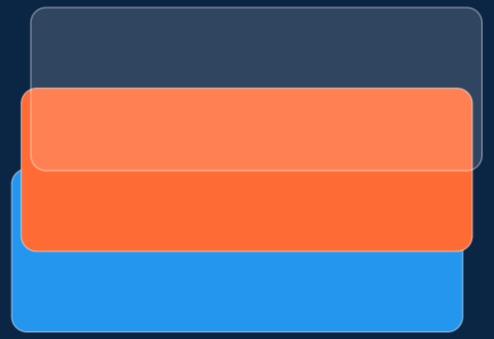
MEAN STACK DEVOPS DEPLOYMENT

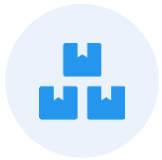
Containerizing a Full-Stack App with Docker and Automating Delivery with a GitHub Actions CI/CD Pipeline



Karthik R
DevOps Engineer | Deployment Case Study
Docker • Docker Compose • Nginx • GitHub Actions • AWS EC2

WHAT WE'LL COVER

Agenda



Project Snapshot

Tech stack and how the pieces fit together

Docker Containerization

Backend & frontend Dockerfiles, multi-stage builds

Docker Compose & Nginx

Orchestrating services, networking, reverse proxy

GitHub Actions CI/CD

Automated build, push and deploy pipeline

Secrets & Infrastructure

AWS EC2 setup and secure credential handling

Findings & Roadmap

Pipeline issues observed and future improvements

PROJECT SNAPSHOT

A Containerized MEAN Application

The application (MongoDB, Express, Angular, Node.js) is split into independent, containerized services — each with a single responsibility — then wired together and shipped through an automated pipeline.

FRONTEND

Angular

User interface

BACKEND

Node.js + Express

REST API

DATABASE

MongoDB

Data persistence

REVERSE PROXY

Nginx

Traffic routing

CONTAINER RUNTIME

Docker

Containerization

ORCHESTRATION

Docker Compose

Multi-container deploy

CI/CD

GitHub Actions

Automated build & deploy

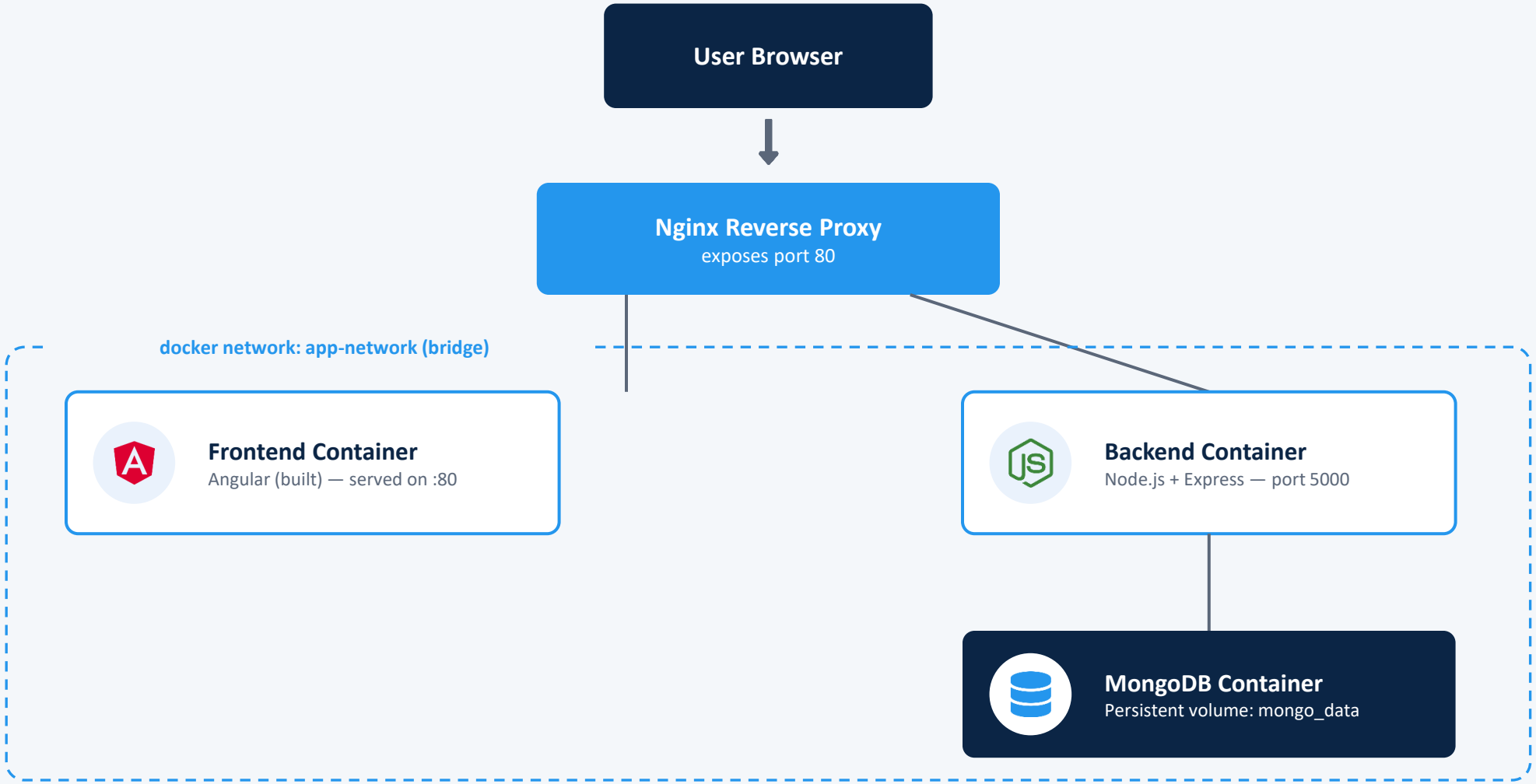
INFRASTRUCTURE

AWS EC2

Cloud hosting

HOW IT FITS TOGETHER

Container Architecture



PART ONE

Docker Containerization

Dockerfiles • Multi-stage builds • Docker Compose • Nginx routing

Backend Image: Node.js + Express

1	`FROM node:18-alpine`	Lightweight official Node base image keeps the image small
2	`WORKDIR /app`	Sets a consistent working directory inside the container
3	`COPY package*.json ./`	Copies dependency manifests first — enables layer caching
4	`RUN npm install`	Installs dependencies in their own cached layer
5	`COPY . .`	Copies the rest of the application source
6	`EXPOSE 5000 / CMD ["npm","start"]`	Documents the API port and defines the startup command

Frontend Image: Multi-Stage Build

Two stages keep the final image small — the Node toolchain used to compile Angular never ships in the production image.

STAGE 1 — BUILD

- `FROM node:18-alpine AS build`
- `WORKDIR /app`
- `COPY package*.json ./  &&  RUN npm install`
- `COPY . .`
- `RUN npm run build --prod`

STAGE 2 — SERVE

- `FROM nginx:alpine`
- `COPY --from=build /app/dist/... /usr/share/nginx/html`
- `COPY nginx.conf /etc/nginx/conf.d/default.conf`
- `EXPOSE 80`

Orchestrating Four Services

mongodb

IMAGE

`mongo:7`

Volume: mongo_data → /data/db

backend

IMAGE

`yashdocker2632/backend:v1.0`

Env: MONGO_URI • depends_on: mongodb

frontend

IMAGE

`yashdocker2632/frontend:v1.0`

depends_on: backend

nginx

IMAGE

`nginx:latest`

Publishes 80:80 • depends_on: frontend, backend

restart: always on every service keeps containers self-healing after a crash or VM reboot. **app-network** (bridge driver) lets containers resolve each other by service name.

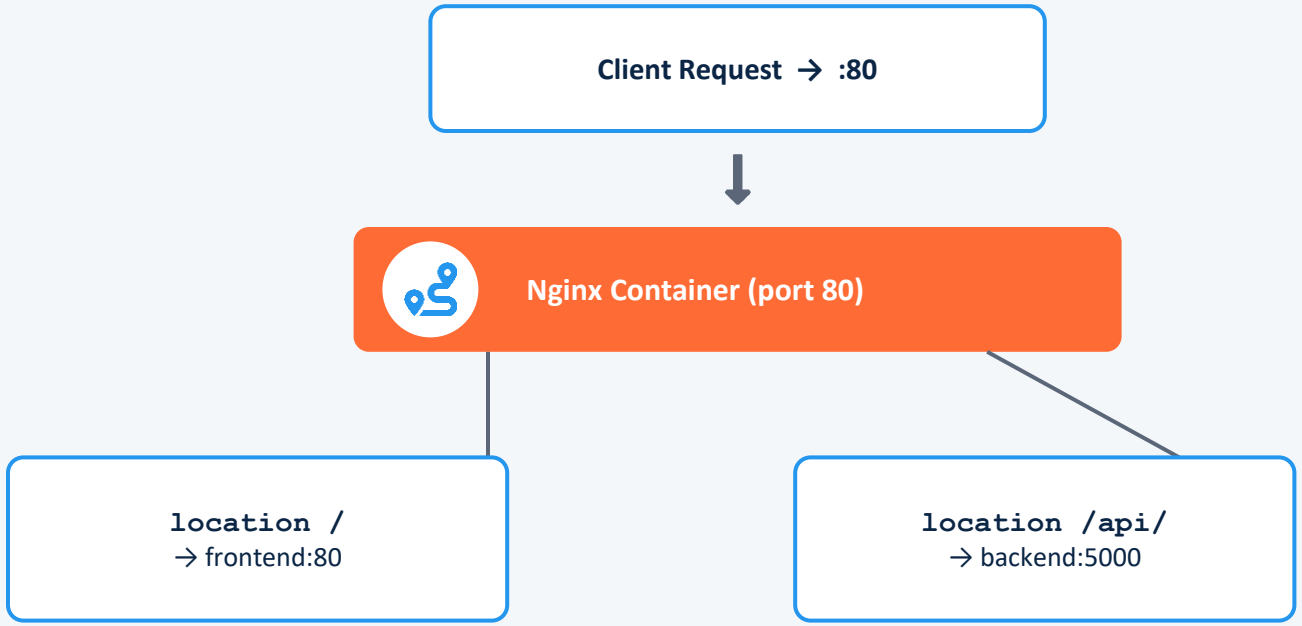
Nginx: Single Entry Point on Port 80

```
server { listen 80; }


location / {
  proxy_pass http://frontend:80;
  }


location /api/ {
  proxy_pass http://backend:5000;
  }
```

proxy_set_header Host / X-Real-IP
preserve the original client info for both routes



One public port hides all internal service ports — the browser never talks to the backend or frontend containers directly.

Provisioning Docker on the EC2 Host

A one-time setup script prepares the Ubuntu VM so the CI/CD pipeline always has a ready Docker + Compose runtime to deploy into.

1. Remove conflicting legacy packages (docker.io, podman-docker, etc.)
2. Update the apt package index
3. Install prerequisites: ca-certificates, curl
4. Create /etc/apt/keyrings directory
5. Download & register Docker’s official GPG key
6. Add the Docker APT repository for the host’s Ubuntu release
7. Refresh apt index with the new repository
8. Install docker-ce, docker-ce-cli, containerd.io, buildx & compose plugins, then start the service

PART TWO

GitHub Actions CI/CD

Trigger • Build • Push to Docker Hub • Deploy over SSH

Pipeline Trigger & Job Structure

Workflow Name

CI-CD Pipeline

TRIGGER

`on: push → branches: [ main ]`

Every commit pushed to main automatically kicks off the pipeline — no manual trigger needed.

JOB

`build-and-deploy`

`runs-on: ubuntu-latest`

A single job runs every step sequentially on a fresh GitHub-hosted runner.

Checkout

Pull the latest commit from the repo

Build & Push

Build both images, push to Docker Hub

Deploy

SSH into the EC2 host, recreate containers

What Each Pipeline Step Does

1

Checkout Code

`actions/checkout@v4`

Clones the repository onto the runner

2

Login to Docker Hub

`docker/login-action@v3`

Authenticates using DOCKER_USERNAME / DOCKER_PASSWORD secrets

3

Build Backend Image

`docker build --no-cache ./backend`

Tags image as <user>/backend:v1.0

4

Build Frontend Image

`docker build --no-cache ./frontend`

Tags image as <user>/frontend:v1.0; --no-cache forces a full Angular rebuild

5

Push Both Images

`docker push (backend, frontend)`

Publishes the freshly built images to Docker Hub

6

Deploy via SSH

`appleboy/ssh-action@v1.0.3`

Connects to the EC2 host and recreates the containers

GITHUB ACTIONS · REPOSITORY → SETTINGS → SECRETS

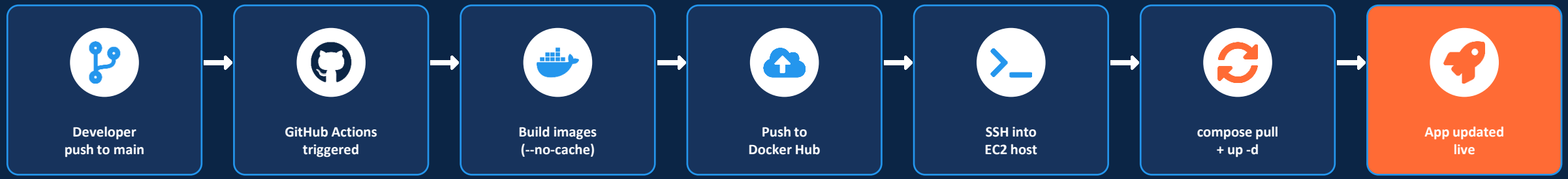
Securing Credentials with GitHub Secrets

The workflow never hard-codes credentials — five encrypted repository secrets are injected at runtime instead.

DOCKER_USERNAME	Docker Hub account used to tag and push images
DOCKER_PASSWORD	Docker Hub authentication token / password
VM_HOST	Public IP of the AWS EC2 deployment target
VM_USER	SSH login user on the EC2 instance
VM_SSH_KEY	Private key used by appleboy/ssh-action to connect

END-TO-END FLOW

From git push to a Running Container



The same pipeline runs on every push — builds are reproducible, image tags stay consistent, and deployment requires zero manual steps on the server.

CRITICAL REVIEW

Pipeline Observations Worth Fixing

Typo breaks the deploy step

The remote script runs "sudp docker compose down" — an invalid command. It will fail on every run and skip the container teardown step.

No rollback on failed SSH deploy

If the SSH/deploy step fails partway, there is no automatic rollback to the last known-good image tag.

--no-cache on every build

Disabling the Docker layer cache guarantees a fresh Angular build, but rebuilds all dependencies every run — slower pipelines than necessary.

Fixed image tag "v1.0"

Both images are always pushed as v1.0 rather than a per-commit tag (e.g. git SHA), making it hard to trace which build is running or roll back precisely.

Secrets handled correctly

Credentials for Docker Hub and the EC2 host are stored as encrypted GitHub Secrets and injected at runtime — nothing is hard-coded in the repo.

Clear separation of concerns

Each container (frontend, backend, mongo, nginx) has a single responsibility, matching Docker best practice.

ROADMAP

Where This Deployment Could Go Next

Kubernetes

Move from Compose to K8s for scaling, self-healing and rolling updates

HTTPS via SSL

Terminate TLS at Nginx (e.g. Let's Encrypt / Certbot) for encrypted traffic

Custom Domain

Point a real domain at the EC2 host instead of a raw IP

Monitoring

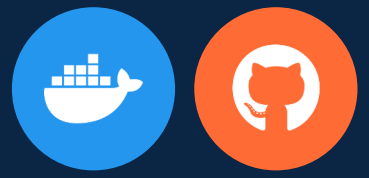
Add Prometheus + Grafana for container and app-level metrics

Infrastructure as Code

Provision the EC2 host and networking with Terraform

Commit-based Tags

Tag images with the git SHA instead of a fixed v1.0 for traceable rollbacks



Thank You

Containerized with Docker • Delivered with GitHub Actions

Karthik R | DevOps Engineer